

由于所有文件数据访问均从 inode 结构出发，因此一个文件中任何位置上的修改，均可通过写时拷贝技术进行原子更新和持久化。

小知识：使用写时拷贝技术保护整个文件系统

除了用来保证文件数据的一致性之外，写时拷贝技术还可以用来保证整个文件系统的一致性^[7]。如在 BtrFS 中，整个存储结构由一棵树组成，可以通过写时拷贝技术保证多个位置上修改的原子完成。

12.4.3 写时拷贝的问题与优化

在上述例子中，我们只希望修改节点 C 和 E 中的部分数据，却导致节点 B、D、A 也被修改。这表现出了写时拷贝的两个问题：更新传递问题和写放大问题。

在树形数据结构中，写时拷贝往往需要更新到根节点才会停止。在图 12.3 中，我们修改节点 C 和 E 后，需要继续修改节点 B、D 和 A（根节点）。这种修改一些节点后，还需要进一步修改数据结构中的其他节点以保证原子更新的问题，被称为更新传递问题。

解决更新传递问题的一种常见方法是提前进行原子更新。对于树形数据结构来说，并非只有指向树根的指针才可以被原子更新；在上述例子中，原子更新的粒度为一个节点。若在更新传递的过程中，所有要进行的修改被一个原子更新粒度所覆盖，则可以直接原地进行修改，无须继续传递更新。例如在图 12.3 中，在节点 A 中保存了指向节点 B 和 D 的指针。由于节点 A 可以被原子更新，我们可以通过一次原地更新操作，原子地修改节点 A 中指向节点 B 和 D 的指针，让它们指向节点 B' 和 D'，从而避免继续使用写时拷贝技术，停止更新传递。

写放大问题是指实际修改数据量大于用户要修改的数据量的情况。如在应用以 4KB 内存页为粒度的写时拷贝技术时，若想修改某个页中的 1 字节数据，我们不得不拷贝整个 4KB 内存页中的所有数据，因此数据修改量从 1 字节被放大到 4KB。写时拷贝技术中的更新传递会使写放大问题更加严重。

当要修改的数据覆盖了拷贝操作的粒度时，可以不拷贝原有数据，直接写入新的数据，从而在一定程度上缓解写放大问题。例如在图 12.3 中，拷贝粒度为一个节点，且节点 E 中的所有数据均需要被修改，在分配 E' 后，即使我们将 E 中的数据拷贝到 E'，这些数据也会被新数据全部覆盖。因此，在这种情况下，可以省略写时拷贝中的拷贝操作，从而减少写入数据量。此外，前面提到的提前原子更新的方法，也可以在一定程度上减轻写放大问题。

12.4.4 讨论和小结

写时拷贝技术对数据结构有一定的要求，当所有的修改最终能够变成一个原子修改

时,才可以使用写时拷贝技术。此外,写时拷贝技术的使用与原子更新粒度和拷贝粒度有关。当修改的数据量远大于拷贝粒度时,往往只有数据头部和尾部真正需要进行拷贝操作,而中间部分可以根据前文提到的方法省略拷贝操作,直接使用新数据写入新分配的节点。这种情况下,写时拷贝技术产生的写放大相对较小。而当数据修改量小于拷贝操作的粒度时,写时拷贝技术造成的写放大会非常严重,在块设备上,即使每次仅修改1字节数据,也需要按照块粒度进行数据拷贝。因此,是否适合使用写时拷贝,需要结合原子更新粒度、拷贝粒度和数据修改量综合考虑。

12.5 Soft updates

本节主要知识点

- 为什么要提出 Soft updates ?
- Soft updates 技术如何保证崩溃一致性?
- Soft updates 技术如何解决环形依赖问题?
- Soft updates 为何无须进行恢复?

虽然使用原子更新技术能够较好地保证文件系统的崩溃一致性,但是在其他方面依然存在一些问题。首先,这些原子更新技术对存储布局和文件系统设计存在一定的要求。例如使用日志时,需要在存储空间中为日志预留空间,同时需要保存日志相关的各种元数据;在写时拷贝技术中,为了方便地通过修改指针原子更新结构内容,每个结构的位置一般由上一级结构中的指针决定,而不能处于固定位置。这限制了文件系统设计的灵活性和可拓展性。此外,使用这些原子更新技术需要在修改过程中对存储设备进行额外的写入操作。例如文件系统使用日志时,需要先将修改的日志记录到存储设备上;在使用写时拷贝时,需要将原来的数据进行一次拷贝操作。这些额外的写入操作反而影响了文件系统的性能。

考虑到这些问题,原子更新技术的使用条件对于一些文件系统来说有些过于严格,影响了文件系统的设计和性能。那么除了原子更新技术之外,有没有其他方法可以保证崩溃一致性呢?

我们不妨回到 12.2.1 节提到的同步写入问题重新进行考虑。为了保证多个修改的持久化顺序,同步写入需要等待前一个修改持久化到存储设备之后,才能开始进行下一个修改。实际上,为了保证崩溃一致性,文件系统仅需保证修改被持久化的顺序,并不需要同步等待每个修改的持久化完成。如果存在某个机制能够保证这些修改的持久化顺序,那么文件系统在内存结构中执行完一个修改之后,无须等待其持久化完成便可以开始处理下一个修改。在这种情况下,文件系统能够以接近内存的性能处理文件请求,并保证

崩溃一致性。

Soft updates^[8-10]就是这样一种通过保证多个修改的持久化顺序来保证文件系统崩溃一致性的机制。Soft updates 技术由 Gregory R. Ganger 等人在 1994 年提出^[9]，其由于良好的性能和对原有文件系统的兼容性，被 FreeBSD 的 UFS 文件系统^[11]、OpenBSD 的 FFS 文件系统^[12]以及 NetBSD^[13]所广泛支持。

12.5.1 Soft updates 的三条规则

Soft updates 通过保证持久化的顺序来保证崩溃一致性，可对于一个具体的包含多个修改步骤的操作来说，到底要如何保证修改的持久化顺序才能避免崩溃一致性被破坏呢？为了回答这个问题，Soft updates 制定了三条规则：

- 规则 1：不要指向一个未初始化的结构。如目录项指向一个 inode 之前，该 inode 结构应该先被初始化。
- 规则 2：当一个结构被指针指向时，不要重用该结构。如当一个 inode 指向了一个数据块时，这个数据块不应该被重新分配给其他结构。
- 规则 3：对于一个仍有用的结构，不要修改最后一个指向它的指针。如重命名文件时，在写入新的目录项前，不应删除旧的目录项。

下面我们通过举例来具体介绍这三条规则。示例中的所有结构均表示在存储设备上保存的结构，其中的操作为对存储设备进行的操作。为了叙述简洁并突出三条规则的作用，下文的讨论对文件操作的过程进行了简化。实际情况会更加复杂，但遵循的规则是相同的。

创建文件

创建文件的过程在 12.1 节已经进行了介绍，主要包括三个步骤：分配 inode，初始化 inode，增加目录项。根据规则 1，inode 初始化操作的持久化应该早于增加目录项操作的持久化，否则增加的目录项会指向一个未初始化的 inode 结构。如果此时系统发生崩溃重启，当文件系统再次通过该目录项访问 inode 结构时，会产生错误。同时，为了保证规则 2 不被破坏，inode 分配同样应该在增加目录项之前持久化。否则，被新增目录项所指向的 inode 结构可能再次被分配和使用，从而导致文件系统产生安全性和正确性问题。

图 12.5 中的创建文件操作创建了一个目录文件。目录文件在创建时需要同时创建“.”和“..”两个目录项，因此其需要分配并初始化一个新的数据块。由于数据块被 inode 结构指向，根据规则 1，数据块的初始化操作应该先于 inode 中索引的修改（包含在 inode 的初始化过程中）进行持久化。同时，由于规则 2，数据块分配信息（如位图）的持久化同样应该在 inode 初始化之前。

由于目录项的大小一般不超过存储设备原子更新的粒度，增加目录项在存储设备上可以被认为是一个原子操作。因此，在文件创建的过程中，在父目录中增加新的目录项可以被认为原子更新点。在此步骤完成之前，所有的操作均不会出现在文件系统树中，

即使发生崩溃，文件系统也只会保留与此文件创建前一致的文件系统树。而在此步骤完成之后，前面各个步骤中修改的结构被“嫁接”到了文件系统树上。由于我们保证此前的步骤均已在存储设备中完成，因此“嫁接”到文件系统树上的这部分“枝丫”同样满足一致性要求。

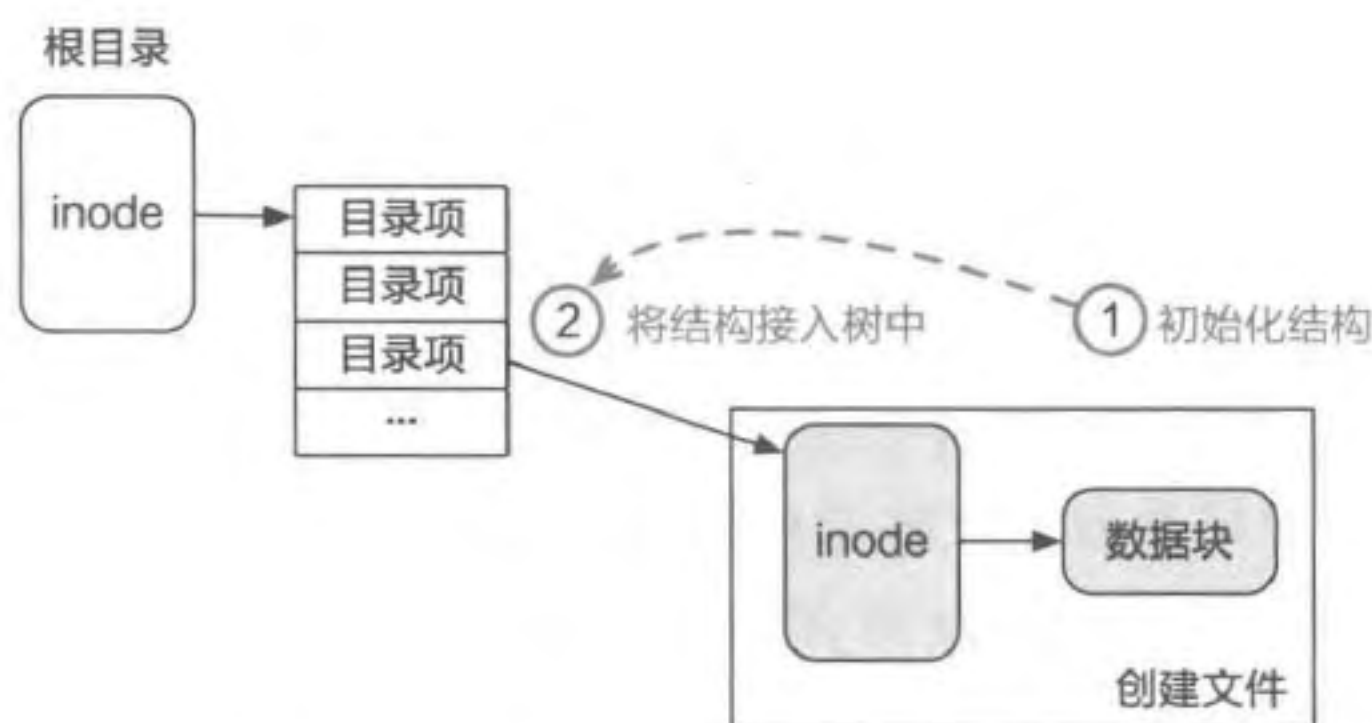


图 12.5 Soft updates 的第一项规则举例

在遵守 Soft updates 规则的情况下，创建文件的过程中依然可能发生空间泄漏问题，即 inode 分配信息被持久化但却未被文件系统使用。但这种情况对文件系统中保存的其他结构不产生影响，可以通过定期扫描找到未使用的 inode 节点并修复。

删除文件

图 12.6 展示了文件的删除操作。常规文件的删除操作需要将文件删除并回收资源。若不考虑硬链接，目录项是目标文件与整个文件系统树的唯一关联点。根据规则 2，文件系统需要将目标文件先从整个文件系统树中“砍”下来，此后再进行资源回收等操作。而这个“砍树枝”的操作即为将对应的目录项从父目录中删除。

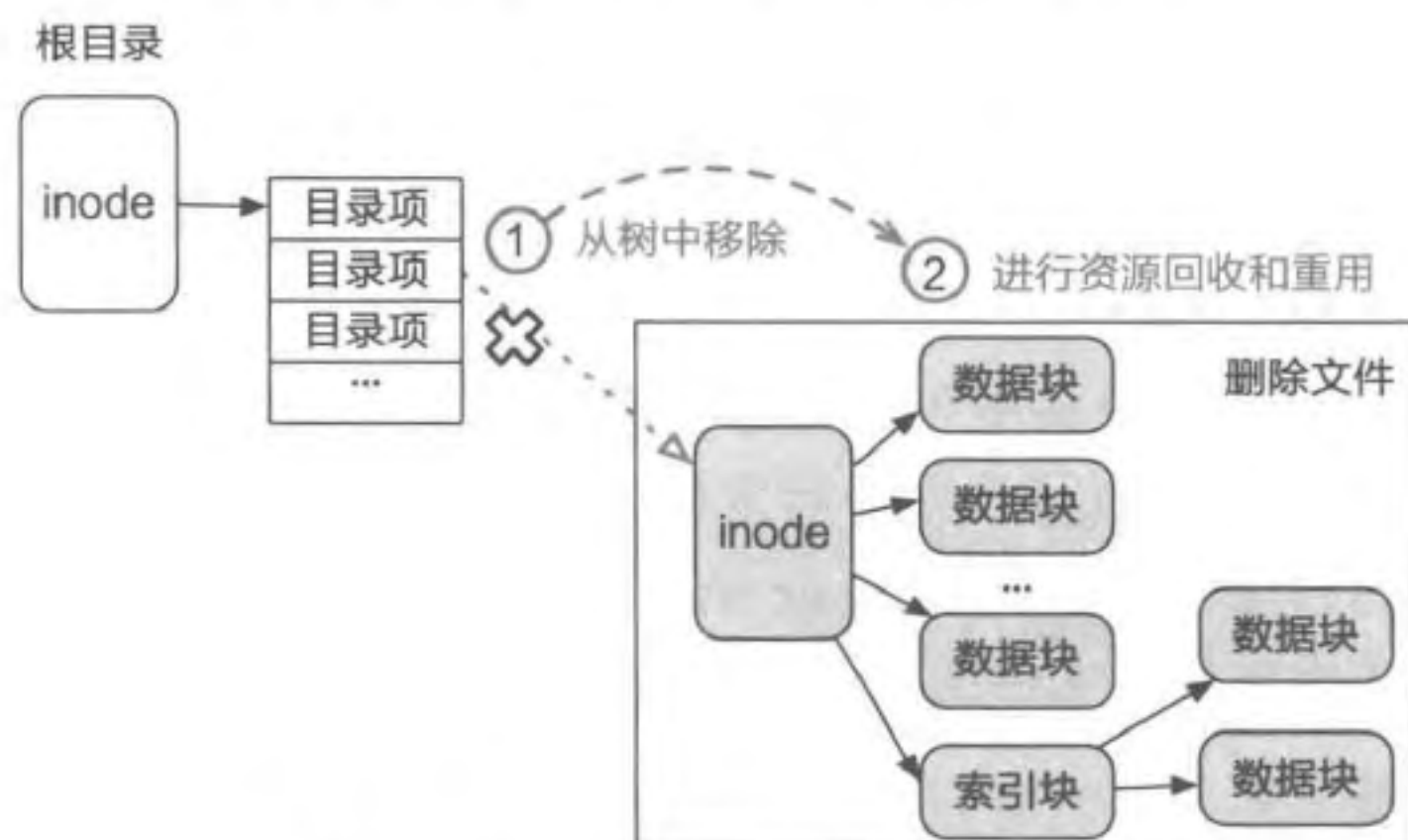


图 12.6 Soft updates 的第二项规则举例

删除目录项为删除操作的原子更新点，系统崩溃只可能发生在此操作之前或之后。若崩溃发生在此之前，则没有删除操作被执行，因此不会产生不一致的情况。若崩溃发

生在此之后，则无论是发生在回收操作之前、之中或者之后，由于这些被回收的资源已经从文件系统树中被移除，这里产生的不一致并不会造成文件系统中其他文件和数据的不一致。此过程中造成的空间泄漏，也可以通过定期检查的方法进行修复。

重命名文件

图 12.7 展示了重命名（rename）操作，其中涉及规则 3 的应用。在重命名操作中，文件系统往往并不进行文件数据的搬移，而是仅仅将指向 inode 的目录项从源目录中删除，并在目标目录中创建指向相同 inode 的新目录项。此过程中新旧目录项中的文件名可以发生改变，从而达到移动文件和重命名文件的效果。

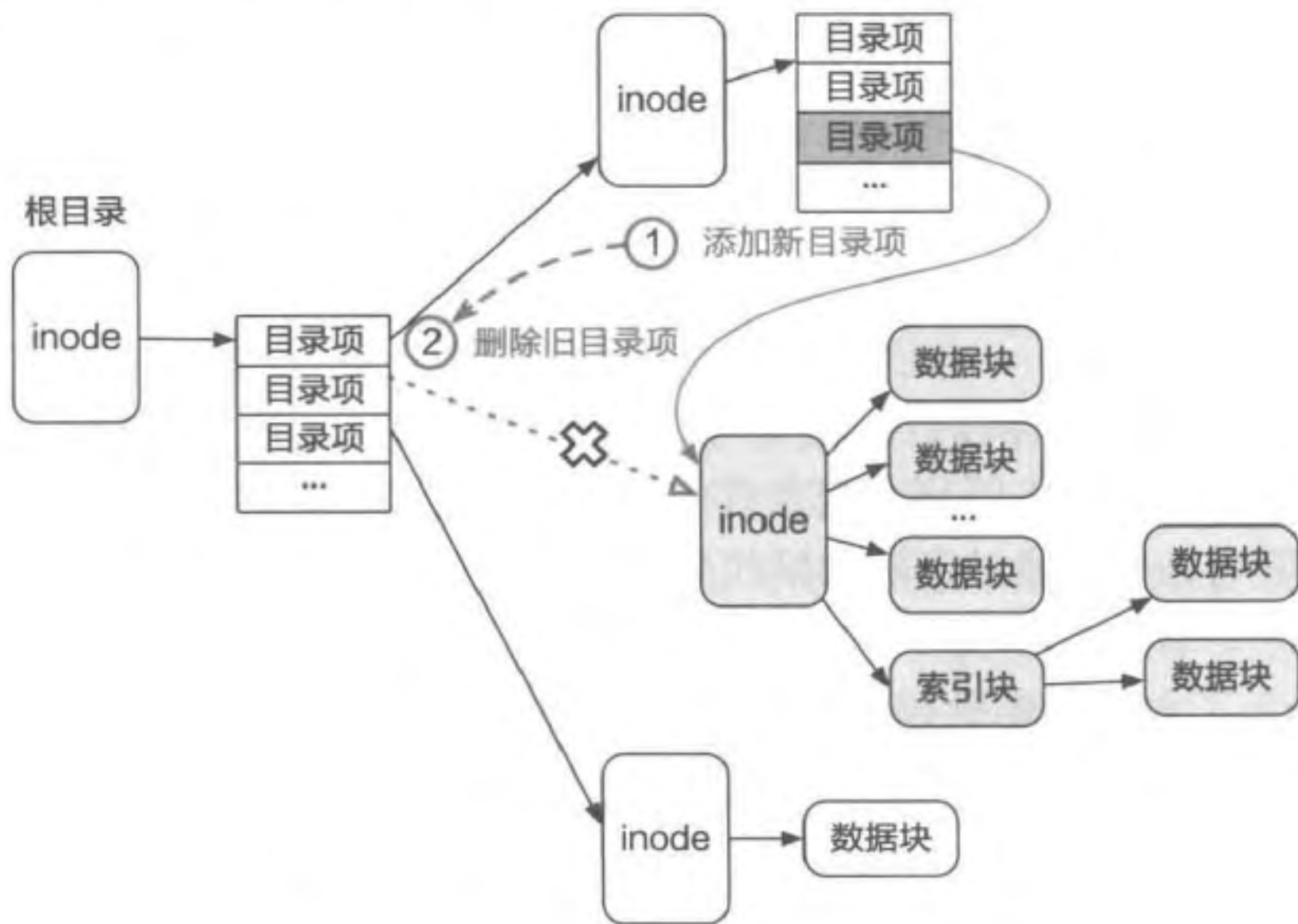


图 12.7 Soft updates 的第三项规则举例

重命名操作涉及的结构和操作比较多，因此更容易产生一致性问题。根据规则 3，Soft updates 在进行文件移动时，需要先保证目标目录中的目录项被写入完毕，之后才能删除源目录中的目录项。

由于目录项修改在存储设备中可以被原子完成，在重命名操作过程中发生的异常情况主要为目标目录中的目录项写入完毕后发生崩溃。在这种情况下，文件系统在重启后会发现两个目录中的目录项均指向该 inode 结构。虽然这种不一致的情况可能会让用户感到惊讶，但其并未造成被移动文件的数据丢失，也不影响文件系统中其他文件的一致性。

12.5.2 依赖追踪

Soft updates 的目标是在进行异步写回的同时，保证多个修改间的持久化顺序，从而在获得高性能的同时，保证文件系统的崩溃一致性。Soft updates 使用内存结构表示还未写回存储设备的修改，并异步地将这些修改持久化到存储设备中。那么 Soft updates 如何

保证这些修改的持久化顺序呢？

为了保证修改的持久化顺序满足上述三条规则，Soft updates 对这些修改之间需要遵守的顺序进行了记录，称为依赖追踪。如果修改 A 需要在修改 B 之前写入存储设备中，则称修改 B 依赖于修改 A。Soft updates 在处理文件系统请求时，除了将修改记录在内存结构中之外，还会将这些修改之间的依赖关系记录下来。

处理依赖关系的一种比较直接的方法是使用有向无环图（Directed Acyclic Graph, DAG）。考虑到文件系统以块为粒度将修改写回存储设备，Soft updates 可以将内存缓存中的块作为 DAG 的顶点，将依赖关系作为边。如当块 A 上的修改需要在块 B 上的修改之前写入存储设备时，图中有一条从 B 指向 A 的边。

图 12.8 给出了基于块粒度的依赖关系示例。示例中为文件创建时的依赖，即在写回新增的目录项（保存在父目录数据块的内存缓存中）前，应该先写回 inode 的分配（保存在 inode 位图所在的数据块缓存中）和初始化（保存在 inode 所在的数据块缓存中）。

然而，这种设计有两个问题：环形依赖（Cyclic Dependency）和写回迟滞（Aging）。由于存储设备的块大小一般远大于 inode 等文件系统结构的大小，因此在一个块中，通常会保存多个文件系统结构。例如，在一个块中会同时保存 inode 表中连续的多个 inode 结构；而在目录的数据块中，会保存目录中的多个目录项结构。这种使用方式会造成环形依赖：块 A 需要在块 B 前写回，同时块 B 需要在块 A 前写回。图 12.9 给出了一个环形依赖的具体例子。其中，文件系统首先分配并初始化 6 号 inode，使其对应 File B，然后新增目录项。在这些修改还未写入存储设备时，文件系统还删除了 File D，对应的 inode 号为 7。这两个操作的目录项刚好在同一个数据块（块 B）中，而涉及的两个 inode 刚好也在同一个块（块 A）中。这时，根据 Soft updates 的持久化规则，在创建文件时，inode 结构的初始化应在新增目录项之前写回，因此块 A 应在块 B 之前被写回存储设备。同时，在删除文件时，目录项的删除应在对 inode 结构进行清理之前写回，因此块 B 应该在块 A 之前写回。一旦产生了这种循环依赖，文件系统无法决定写回顺序。除了循环依赖的问题之外，当一个结构中的数据被频繁修改时，该结构很可能由于一直产生新的依赖而导致长时间无法被写回存储设备中，即写回迟滞问题。

12.5.3 撤销和重做

为了解决循环依赖和写回迟滞的问题，Soft updates 将依赖追踪从块粒度细化为结构粒度，并使用了撤销（Undo）和重做（Redo）方法来打破循环依赖。在进行依赖追踪时，Soft updates 会记录每个结构上的修改记录。当需要将某个结构写回存储设备时，Soft updates 会查看该结构的依赖关系，检测是否有环形依赖产生。当出现环形依赖时，先将部分操作撤销（即将内存中的结构还原到此操作执行前的状态）。在撤销之后，环形依赖被打破，此时可以根据打破后的依赖将修改按照顺序进行持久化。持久化完毕后，Soft updates 需要将此前被撤销的操作恢复，即重做。在重做完成后，Soft updates 可以将最

新的内存中的结构按照新的依赖关系再次进行持久化。

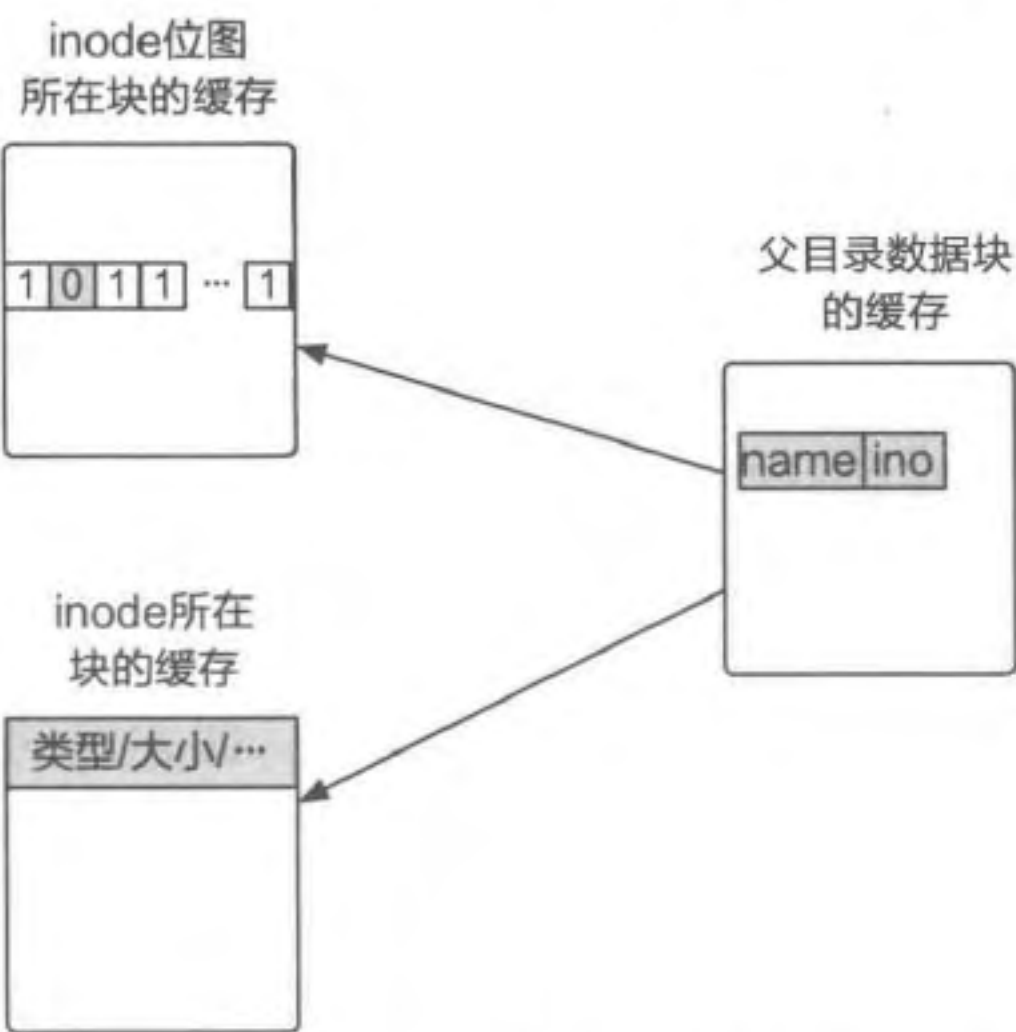


图 12.8 Soft updates 基于块的 DAG 依赖关系

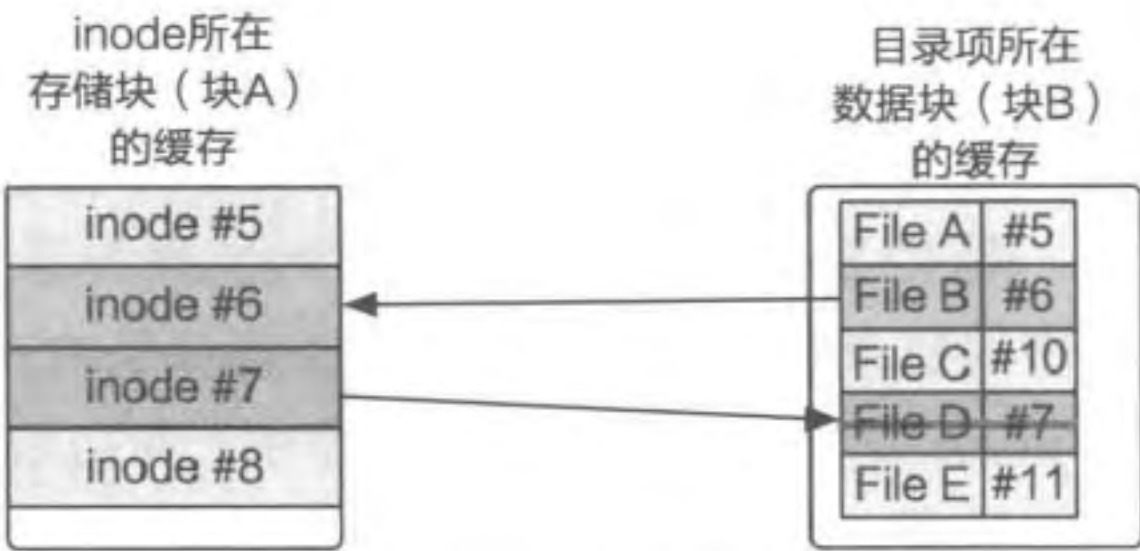


图 12.9 Soft updates 的环形依赖问题^[8]

图 12.10 给出了使用撤销和重做方法打破循环依赖的示例。对于此前创建 File B 和删除 File D 造成的环形依赖，Soft updates 首先撤销删除 File D 的修改（①）。在撤销完毕后，块 A 和块 B 之间只剩下一个依赖关系。只需要按照依赖关系先将块 A 写回，再将块 B 写回即可（②）。在修改持久化到存储设备后，Soft updates 重做刚刚撤销的删除操作（③）。重做后块 A 和块 B 之间依然只有简单的依赖关系，此时继续按照顺序进行写回即可。在撤销到重做期间，Soft updates 通过对结构上锁，避免应用程序错误地看到撤销后的结构状态。

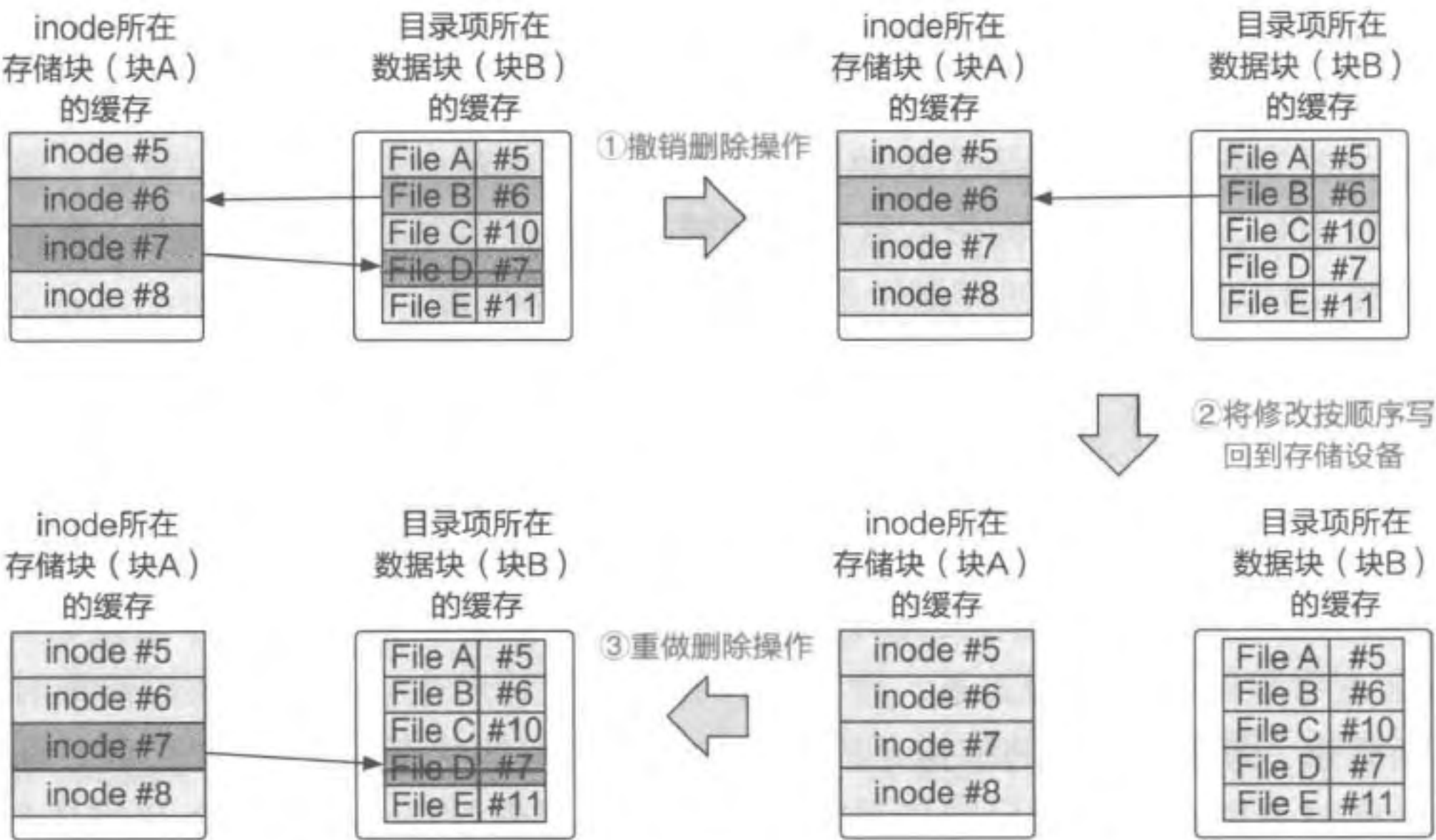


图 12.10 Soft updates 使用撤销和重做打破环形依赖^[8]

这种撤销、持久化、重做的方式不仅能够打破环形依赖、保证持久化顺利进行，还能解决写回迟滞的问题。若某个结构被频繁修改，导致不断有新的依赖产生，Soft updates 可以将部分新的修改撤销，在快速完成持久化后将修改重做，以此避免新依赖不断推迟该结构上修改的持久化。

12.5.4 文件系统恢复

在发生崩溃后，文件系统一般需要通过 12.2 节介绍的文件系统检查 (fsck) 来保证文件系统的一致性。在 fsck 完成之前，文件系统一般处于无法使用状态，以保证潜在的非一致性问题不会造成更加严重的数据损坏。然而 fsck 所需要的时间与文件系统的大小相关。这将导致在崩溃之后的很长一段时间内，文件系统都处于不可用状态。对于 Soft updates 来说，发生崩溃之后同样有可能产生不一致。但是由于 Soft updates 在写回时严格按照依赖顺序，在崩溃后，Soft updates 文件中存在的不一致并不会导致文件系统无法使用或导致其中的数据发生损坏。一些可能发生的不一致情况包括：

- 未使用的块没有出现在空闲空间列表中。
- 未被引用的 inode 没有出现在空闲 inode 列表中。
- inode 中保存的链接数可能超过实际链接数（如 inode 中链接数为 3，却只有 2 个目录项指向此 inode），从而导致该 inode 及其引用的数据块无法被回收。

即使发生了这些不一致情况，Soft updates 依然可以在挂载后立即开始处理新的文件系统请求，无须等待耗时的文件系统检查。管理员可以选择在合适的时刻执行 Soft updates 的文件系统检查操作来修复这些问题。

12.5.5 讨论和小结

总的来说，Soft updates 通过记录和保持文件系统结构之间的持久化依赖关系，保证各个结构按照既定的顺序进行持久化，从而在异步写回的情况下避免在存储设备上产生严重的不一致情况。异步写回使得 Soft updates 能够以内存的性能完成文件系统操作；而严格的写回（持久化）顺序令 Soft updates 在崩溃之后可以立即投入使用，无须等待长时间的恢复或者检查。对于可能出现的一些“良性”不一致情况，用户可以在适当的时刻通过文件系统检查进行修复，以此来保证整个文件系统的一致性。

Soft updates 是不同于日志、写时拷贝等技术的保证文件系统一致性的方法，其设计上避免了对文件系统存储格式的修改，使得文件系统的设计与一致性保证解耦。同时，其避免了日志的两次写入和写时拷贝的写放大等性能问题。然而，Soft updates 也并非完美的。为了实现 Soft updates 中涉及的各种机制，文件系统需要在内存中记录大量的依赖关系。这会用到大量的辅助结构，导致 Soft updates 的实现非常复杂。因此，在实际使用中，Soft updates 还经常与日志等机制结合起来使用^[14]，以简化系统的设计与实现。

12.6 案例分析：日志结构文件系统

本节主要知识点

- 日志结构文件系统是如何保存文件的数据和元数据的？
- 日志结构文件系统如何管理空间？
- 日志结构文件系统与使用日志的文件系统有何区别？有何优势？
- 日志结构文件系统如何保证文件系统崩溃一致性？

本章在介绍重做日志时，曾提到重做日志不急于将日志内容写回原位置。那么是否能够更极端一些，直接将整个文件系统以一种日志的形式保存呢？这样每个文件系统结构都没有“原位置”，也就不需要对日志内容进行写回操作了。日志结构文件系统（Log-structured File System, LFS）正是这种主要以日志结构保存数据和元数据的文件系统。在本节中，我们将基于最早的 Sprite LFS^[15]对日志结构文件系统的基本概念和原理进行介绍。

小知识：最初 LFS 设计的假设

最初的 LFS 设计基于这样一个假设：由于文件系统数据可以被缓存在内存中，大多数文件系统读请求可以在内存中完成。随着内存容量的提升，文件系统的读取操作可以达到比较好的性能。在这种情况下，对于存储设备的访问将主要被写请求占据。考虑到机械硬盘顺序写入可以避免磁盘寻道时间，以日志的形式组织文件系统可以让写入性能达到最优。另外需要注意的是日志与日志结构这两种技术是有区别的。日志一般是临时性的，日志中记录的内容最终会被写入它们原本应该在的位置；而在使用日志结构时，数据并没有固定的保存位置。

12.6.1 基本概念与空间布局

在此前介绍的文件系统中，每个结构在存储设备上均有固定的存储位置。当某个结构中的数据被修改时，其修改被“原地”写入，该结构的位置并不会发生变化。当多个结构被修改时，若这些结构的固定位置不能被一个原子修改操作覆盖，文件系统需要向存储设备发出多次随机写入请求。由于存储设备的随机写入性能一般要比顺序写入性能差，大量的随机写入请求会降低文件系统的总体性能。

为了减少随机写入，LFS 以日志的形式保存整个文件系统的数据。所有的修改均会追加到日志空间的尾部，从而保证对于存储设备的顺序写入。由于使用这种写入方式，日志中保存的文件结构和数据并没有固定的“原位置”。随着文件系统的使用，每个结构的保存位置也会不断发生变化。那么如何才能在不断变化位置的结构中找到某个特定的